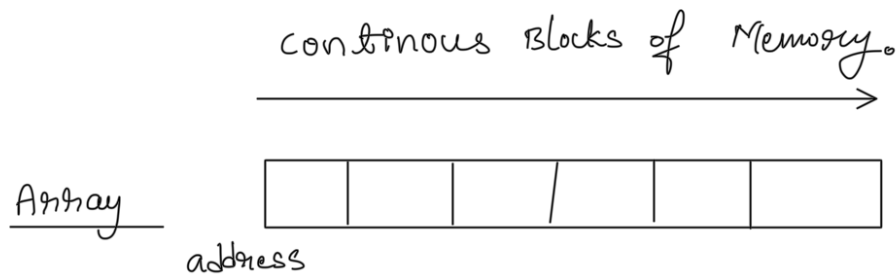


BEHIND THE SCENES OF LINEAR DATASTRUCTURE (Array & linked list as Example)

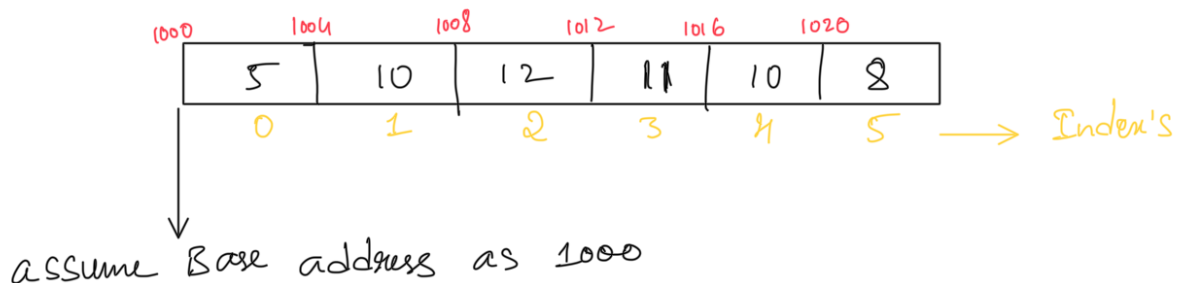
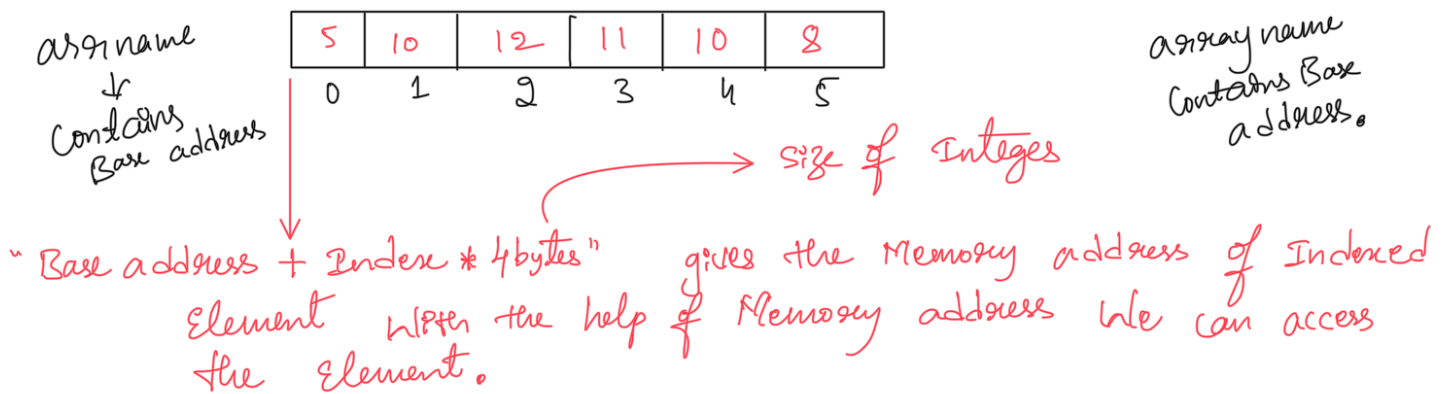
Day 33
12/08/2025



The contiguous Memory Locations of Array store Homogenous data type elements. All the elements present in the Array are of Same type.

Array Have Concept of Index. (In C & C++)

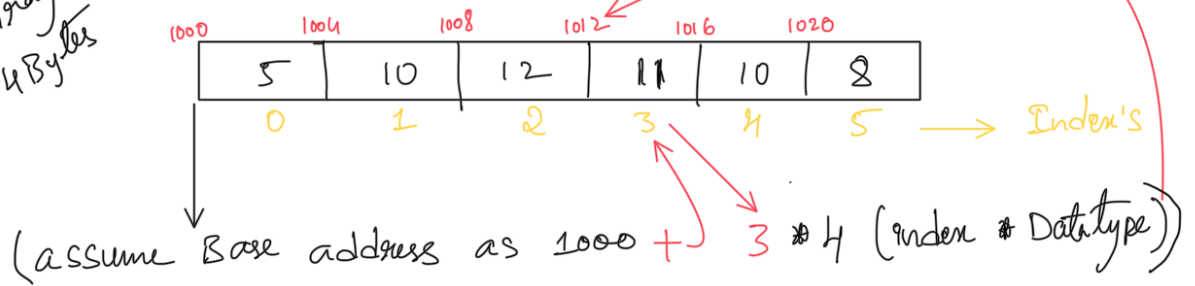
Assume we have array of Integer data type



How CPU reaches the i^{th} position indexed element in $O(1)$ time complexity?

(array Base Address + datatype * Index) gives memory location which gets the direct access to element in constant time $O(1)$.

Integer array
size = 4 Bytes



if I want to access 3rd Index element
the Memory address will be

$$1000 + (3 * 4) = 1,012$$

1012 is the Memory Location where the 3rd indexed element is stored.

The $O(1)$ access of array elements reduces the CPU cost

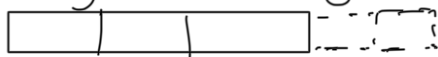
The Homogeneous elements of any data type can be stored inside the Array (In JAVA)

array can store → primitive Datatypes
→ Non primitive Datatypes

In JAVA "ARRAYS" stores the objects also as elements

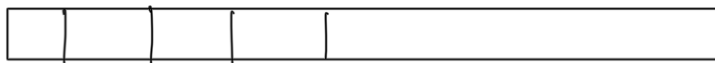
Disadvantages of Array.

① Array is not Dynamic Data structure



We cannot add elements more than its size

②



Wasting Memory
from RAM

↑ Not using full array size
Some times

Arrays can create more complications Bcz of its fixed size

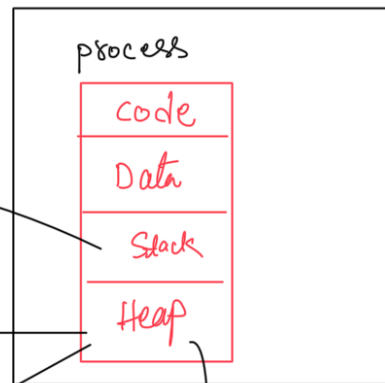
So Here Linked List came into picture which includes features of Dynamic Memory Allocation (DMA)

DMA

Local Variables are Declared in stack Memory.

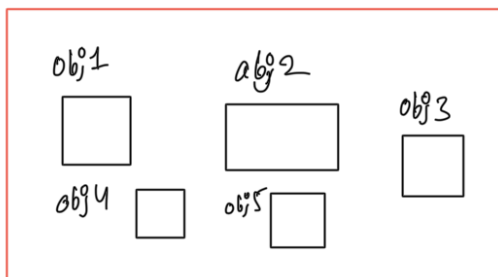
objects are stored in Heap Memory

RAM



Heap Memory supports Dynamic Memory Allocation.

Heap



If Heap memory gets occupied by more numbers of objects then it gives out of memory Exception.

To avoid getting this Exception we have to free space of unused objects (objects which no other element is referring). The objects with 0 references are deleted from Heap Memory.

In C/C++ the object deletion is done manually using free keyword.

→ "malloc / calloc" for Dynamic Memory Allocation

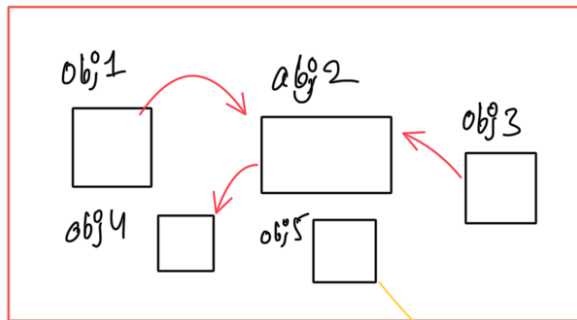
→ "free" for deletion of allocated memory

Manual deletions created so many complications

To overcome this **garbage collector** is created.

Search for the objects which are not used or having no references, & are being deleted by the garbage collector to free up memory.

Heap

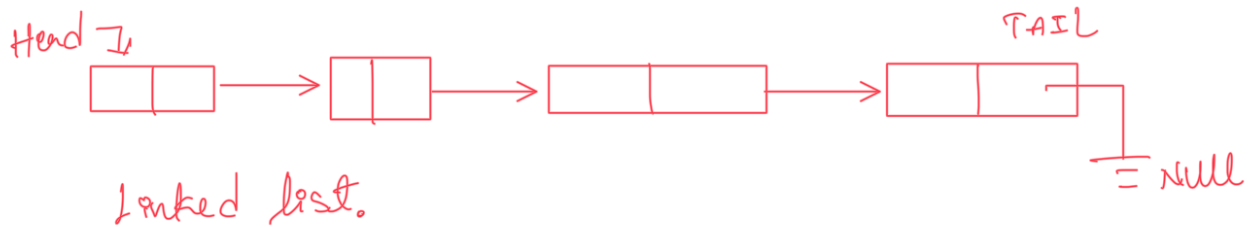
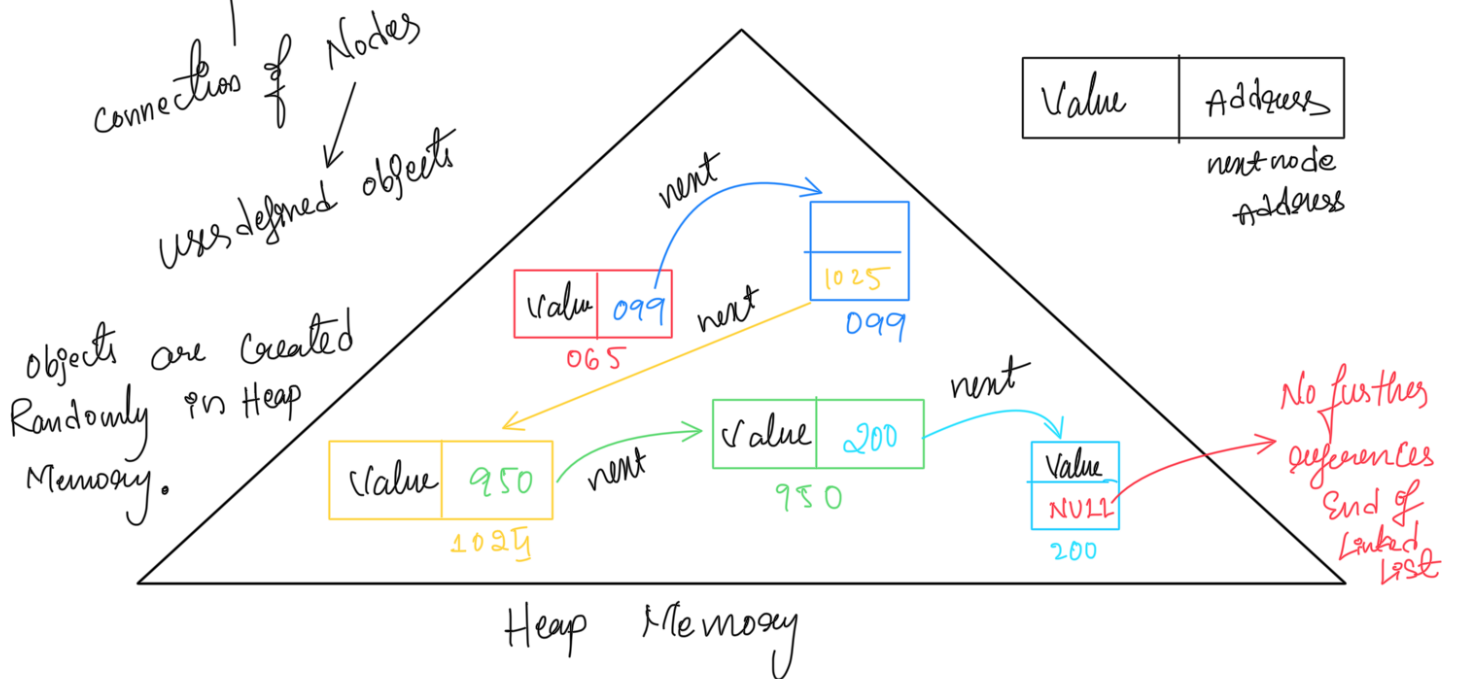


Some times the programmer forgot to clear reference for the unused objects. But the garbage collector thinks someone is using it and that object is never deleted. This causes **Memory Leak**.

Memory Leak

Key Differences		
Feature	Memory Leak	Out of Bounds Exception
Cause	Keeping unused objects referenced	Accessing invalid array/list index
Error Type	May lead to <code>OutOfMemoryError</code>	Runtime exception (<code>IndexOutOfBoundsException</code>)
GC Involvement	Yes (blocked cleanup)	No (bounds check error)
Detection	Usually via monitoring/profilers	Exception thrown immediately
Fix	Remove unnecessary references	Check index bounds before access
💡 Think of it like:		
• Memory Leak → Your backpack keeps filling because you never throw old stuff away.		
• Out of Bounds → You're reaching into a pocket that doesn't exist.		

Linked List (Connected Nodes) (Nodes are user defined data type)



The Linked List supports Dynamic Memory Allocation & it's a Linear Data Structure which helps to overcome the array disadvantages.

Further I am going to Explore this Linear Data Structures with Implementations.